

Practical No:6

Aim:

K-Nearest Neighbors (K-NN)

Implement the K-NN algorithm for classification or regression.

Apply the K-NN algorithm to a given dataset and predict the class or value for test data.

Evaluate the accuracy or error of the predictions and analyze the results.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt

df = pd.read_csv("iris.csv")
df = df.drop(columns=["Id"])

X = df.drop(columns=["Species"])
y = df["Species"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

k_range = range(1, 21)
accs = []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_s, y_train)
    accs.append(accuracy_score(y_test, knn.predict(X_test_s)))

best_k = list(k_range)[accs.index(max(accs))]
print(f"Best k: {best_k}, Accuracy: {max(accs):.4f}")
print()

model = KNeighborsClassifier(n_neighbors=best_k)
model.fit(X_train_s, y_train)
```

```
y_pred = model.predict(X_test_s)

print(f"Test Accuracy (k={best_k}): {accuracy_score(y_test, y_pred):.4f}")
print()
print("Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print()

for k in [1, 3, 5, 7, 9, 11, 15]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_s, y_train)
    acc = accuracy_score(y_test, knn.predict(X_test_s))
    err = 1 - acc
    print(f"k={k}: accuracy={acc:.4f}, error_rate={err:.4f}")

plt.figure(figsize=(8, 5))
plt.plot(list(k_range), accs, marker="o")
plt.axvline(x=best_k, color="r", linestyle="--", label=f"Best k={best_k}")
plt.xlabel("k (Number of Neighbors)")
plt.ylabel("Test Accuracy")
plt.title("KNN Accuracy vs k")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("knn_k_selection.png", dpi=150)
print("Saved knn_k_selection.png")
```

